

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ

По лабораторной работе №4

По дисциплине «Логика и основы алгоритмизации в инженерных
задачах»

На тему «Бинарное дерево поиска»

Выполнил студент группы
24ВВВ2:

Тремасов К. И.

Приняли:

Юрова О.В.

Деев М.В.

Пенза, 2025

Цель работы

Научиться реализовывать алгоритм бинарное дерево поиска, осуществлять поиск элементов и выводить дерево корректно на экран.

Лабораторное задание

1. Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.
2. Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.
3. Изменить функцию добавления элементов для исключения добавления одинаковых символов.
4. Оценить сложность процедуры поиска по значению в бинарном дереве.

Описание метода решения задачи

Для реализации алгоритма двоичного дерева его можно представить в виде нелинейного связанного списка (значение левого потомка меньше значения родителя, значение правого потомка больше значения родителя)

Из-за специфики языка, в решении я буду реализовывать рекурсию в функциях `InsertRec`, `ContainsRec`, `PrintTreeRec` и буду это делать без указателей, вместо этого используя ссылки и объекты. Очистка памяти и ее выделение не потребуются.

Сложность алгоритма бинарного дерева поиска

Сложность поиска в бинарном дереве поиска:

Лучшая: $O(1)$, если искомое значение находится в корне дерева;

Средняя: $O(\log n)$, если дерево сбалансировано.");

Худшая: $O(n)$, если дерево вырождено (в виде списка).");

Листинг программы

```
using System;

public class TreeNode<T>
{
    public T Value { get; set; }
    public TreeNode<T> Left { get; set; }
    public TreeNode<T> Right { get; set; }

    public TreeNode(T value)
    {
        Value = value;
        Left = null;
        Right = null;
    }
}

public class BinarySearchTree<T> where T : IComparable<T>
{
    public TreeNode<T> Root { get; private set; }
    private int count = 0;
    private const int MaxElements = 20;
    private readonly bool allowDuplicates;

    public BinarySearchTree(bool allowDuplicates)
    {
        this.allowDuplicates = allowDuplicates;
    }

    public bool Insert(T value)
    {
        if (count >= MaxElements)
        {
            Console.WriteLine("Превышено максимальное количество элементов (20).");
            return false;
        }
    }
}
```

```

    }

    bool inserted = false;
    Root = InsertRec(Root, value, ref inserted);

    if (inserted) count++;
    else if (!allowDuplicates)
        Console.WriteLine("Значение уже существует в дереве и не будет
добавлено.");

    return inserted;
}

private TreeNode<T> InsertRec(TreeNode<T> node, T value, ref bool inserted)
{
    if (node == null)
    {
        inserted = true;
        return new TreeNode<T>(value);
    }

    int comparison = value.CompareTo(node.Value);
    if (comparison < 0)
    {
        node.Left = InsertRec(node.Left, value, ref inserted);
    }
    else if (comparison > 0)
    {
        node.Right = InsertRec(node.Right, value, ref inserted);
    }
    else if (allowDuplicates)
    {
        node.Right = InsertRec(node.Right, value, ref inserted);
    }

    return node;
}

```

```

public bool Contains(T value)
{
    return ContainsRec(Root, value);
}

private bool ContainsRec(TreeNode<T> node, T value)
{
    if (node == null) return false;

    int comparison = value.CompareTo(node.Value);
    if (comparison == 0) return true;

    return comparison < 0 ? ContainsRec(node.Left, value) :
ContainsRec(node.Right, value);
}

public int CountOccurrences(T value)
{
    return CountOccurrencesRec(Root, value);
}

private int CountOccurrencesRec(TreeNode<T> node, T value)
{
    if (node == null) return 0;

    int comparison = value.CompareTo(node.Value);
    if (comparison == 0)
    {
        return 1 + CountOccurrencesRec(node.Right, value);
    }
    else if (comparison < 0)
    {
        return CountOccurrencesRec(node.Left, value);
    }
    else
    {
        return CountOccurrencesRec(node.Right, value);
    }
}

```

```

    }

    public void PrintTree()
    {
        if (Root == null)
        {
            Console.WriteLine("[пусто]");
        }
        else
        {
            PrintTreeRec(Root, "", false, true);
        }
    }

    private void PrintTreeRec(TreeNode<T> node, string indent, bool isRight,
bool isRoot)
    {
        if (node == null)
            return;

        string newIndent = indent + "    ";

        if (node.Right != null)
            PrintTreeRec(node.Right, newIndent, true, false);

        Console.Write(indent);
        if (!isRoot)
            Console.Write(isRight ? "└─ " : "┌─ ");
        Console.WriteLine(node.Value);

        if (node.Left != null)
            PrintTreeRec(node.Left, newIndent, false, false);
    }
}

class Program

```

```

{
    static void Main()
    {
        Console.Write("Разрешить ввод повторяющихся значений? (да/нет): ");
        string allowDuplicatesInput = Console.ReadLine().Trim().ToLower();
        bool allowDuplicates = allowDuplicatesInput == "да";

        BinarySearchTree<int> tree = new
BinarySearchTree<int>(allowDuplicates);

        Console.WriteLine("\nВведите до 20 чисел для вставки в дерево. Для
завершения введите пустую строку.");

        while (true)
        {
            Console.Write("Введите число: ");
            string input = Console.ReadLine();

            if (string.IsNullOrEmpty(input)) break;

            if (int.TryParse(input, out int value))
            {
                tree.Insert(value);
            }
            else
            {
                Console.WriteLine("Некорректный ввод.");
            }
        }

        Console.WriteLine("\nДерево (повернутое на 90 градусов, корень
слева):");
        tree.PrintTree();

        Console.Write("\nВведите значение для поиска: ");
        if (int.TryParse(Console.ReadLine(), out int searchVal))
        {
            bool found = tree.Contains(searchVal);

```

```

        Console.WriteLine(found ? "Значение найдено." : "Значение не
найдено.");
    }

    if (allowDuplicates)
    {
        Console.Write("\nВведите значение для подсчета вхождений: ");
        if (int.TryParse(Console.ReadLine(), out int countVal))
        {
            int occurrences = tree.CountOccurrences(countVal);
            Console.WriteLine($"Количество вхождений: {occurrences}");
        }
    }
    else
    {
        Console.WriteLine("\nПодсчёт количества вхождений отключен
(дубликаты не разрешены).");
    }

    Console.WriteLine("\nСложность поиска в бинарном дереве поиска:");
    Console.WriteLine("- Средняя:  $O(\log n)$ , если дерево сбалансировано.");
    Console.WriteLine("- Худшая:  $O(n)$ , если дерево вырождено (в виде
списка).");
}
}

```


Результат работы программы

```
Выбрать C:\Users\ardon\source\repos\lab4cs\lab4cs\bin\Debug\lab4cs.exe
Разрешить ввод повторяющихся значений? (да/нет): нет

Введите до 20 чисел для вставки в дерево. Для завершения введите пустую строку.
Введите число: 235
Введите число: 34
Введите число: 32
Введите число: 68
Введите число: 46
Введите число: 75
Введите число: 3
Введите число: 80
Введите число: 68
Значение уже существует в дереве и не будет добавлено.
Введите число: 6
Введите число: 4
Введите число: -24
Введите число: -4
Введите число:

Дерево (повернутое на 90 градусов, корень слева):
235
  └── 80
      ├── 75
      │   ├── 68
      │   │   ├── 34
      │   │   └── 32
      │   └── 46
      └── 6
          └── 4
              └── 3
```

```
Консоль отладки Microsoft Visual Studio
Введите число: -24
Введите число: -4
Введите число:

Дерево (повернутое на 90 градусов, корень слева):
235
  └── 80
      ├── 75
      │   ├── 68
      │   │   ├── 34
      │   │   └── 32
      │   └── 46
      └── 6
          ├── 4
          └── 3
              └── -4
                  └── -24

Введите значение для поиска: 75
Значение найдено.

Подсчёт количества вхождений отключен (дубликаты не разрешены).
```

```
Консоль отладки Microsoft Visual Studio
Разрешить ввод повторяющихся значений? (да/нет): да

Введите до 20 чисел для вставки в дерево. Для завершения введите пустую строку.
Введите число: 13
Введите число: 42
Введите число: 36
Введите число: 76
Введите число: -1
Введите число: -1
Введите число: -1
Введите число: 5
Введите число: 2
Введите число:

Дерево (повернутое на 90 градусов, корень слева):
13
  └── 76
      ├── 42
      │   └── 36
      └── 5
          └── 2
              └── -1
                  ├── -1
                  └── -1

Введите значение для поиска: 24
Значение не найдено.

Введите значение для подсчета вхождений: -2
Количество вхождений: 0
```

Вывод: Реализовали алгоритм, включающий в себя алгоритм бинарного дерева поиска, подсчёт количества вхождений элемента в дерево, имплементировали возможность ограничить вводимые числа, исключив повторения в бинарном дереве. Научились реализовывать алгоритм бинарного дерева.